

HOJA DE REFERENCIA MIPS 32

Arquitectura del Computador.

Penélope Robledo.

Sección 1.

CONCEPTOS GENERALES

Estructura de un Archivo Ensamblador: Todo programa se divide en dos secciones principales mediante directivas:

- **.data:** Aquí se declaran los datos. Físicamente se alojan las variables en RAM, mensajes, cadenas de texto (**.ascii**) y arreglos.
- **.text:** Aquí reside el código. Se escribe la lógica del programa, las instrucciones a ejecutar directamente por el procesador, y la etiqueta de inicio **main**.

Ejemplo de una Sintaxis Estructural Inicial:

```
.data
    numero: .word 25          # Variable entera comun
    texto: .ascii "Hola"     # Cadena de texto (String)

.text
main:
    lw $t0, numero           # Inicia el código cargando un valor
```

Organización de una Línea de Instrucción MIPS:

- **OPCODE / INSTRUCCIÓN:** Especifica la operación a realizar (ej: 'add').
- **REGISTRO DESTINO:** El registro donde se almacenará el resultado.
- **OPERANDOS:** Los elementos de origen desde donde saldrá la información.

Principales Registros (De un total de 32):

- **\$t0-\$t9:** Registros temporales para almacenar operaciones o contadores de corto plazo.
- **\$a0-\$a3:** Paso de argumentos requeridos para rutinas y el sistema.
- **\$v0-\$v1:** Valores retornados por funciones y códigos que solicitan un **syscall**.
- **\$ra:** Dirección de retorno requerida al utilizar un salto tipo **jal**.

A. Operaciones Aritméticas

1. ADD (Suma de Registros)

Instrucción: Suma los valores contenidos en dos registros de origen y almacena el resultado en un registro destino.

```
add $t0, $t1, $t2    # Sumar registros
```

Explicación: Se toman los valores numéricos almacenados en los registros temporales **\$t1** y **\$t2**, se suman, y el resultado final se guarda en el registro **\$t0**.

2. ADDI (Suma con Inmediato)

Instrucción: Suma el valor de un registro de origen con un valor constante (un número directo escrito en el código) y guarda el resultado en un registro destino.

```
1 addi $s0, $s1, 100 # Sumar constante
```

Explicación: Se toma el valor contenido en el registro `$s1`, se le suma directamente la constante numérica 100, y el resultado se almacena en el registro `$s0`.

3. SUB (Resta de Registros)

Instrucción: Resta el contenido del segundo registro de origen al primer registro de origen y guarda la diferencia en un registro destino.

```
1 sub $t0, $t1, $t2 # Restar registros
```

Explicación: Se realiza la operación matemática de restar el valor del registro `$t2` al valor del registro `$t1` (`$t1 - $t2`). El resultado obtenido se almacena en `$t0`.

4. SUBI (Resta con Inmediato)

Instrucción: Resta un valor numérico constante directamente al contenido de un registro de origen y guarda el resultado en un registro destino.

```
1 subi $s0, $s1, 50 # Restar constante numerica
```

Explicación: Se le resta el número constante 50 al valor actual del registro `$s1`, y el resultado final de la resta se guarda en el registro `$s0`.

B. Transferencia de Datos

5. LI (Cargar Inmediato)

Instrucción: Introduce un número entero constante directamente dentro de un registro.

```
1 li $t0, 50 # Cargar numero directo
```

Explicación: Se asigna el número entero 50 de forma directa en el registro `$t0`, inicializando dicha variable con ese valor.

6. LA (Load Address)

Instrucción: Carga la dirección de memoria de una etiqueta declarada previamente en la sección de datos (`.data`).

```
1 la $a0, mensaje # Cargar la direccion del texto
```

Explicación: Se busca la ubicación en la memoria RAM donde se encuentra el dato llamado `mensaje` y se guarda esa dirección de memoria dentro del registro `$a0`.

7. MOVE (Copiar)

Instrucción: Duplica el valor contenido en un registro de origen y lo coloca en un registro destino.

```
1 move $t0, $t1      # Copiar contenido de $t1 en $t0
```

Explicación: Se toma el valor que se encuentra dentro del registro `$t1` y se copia en el registro `$t0`. El valor original de `$t1` permanece intacto.

8. LW (Load Word)

Instrucción: Copia un dato de 32 bits (una palabra completa) desde una dirección específica de la memoria RAM hacia un registro.

```
1 lw $t1, 0($s0)     # Traer los bits desde memoria a t1
```

Explicación: Se accede a la posición de memoria RAM apuntada por el registro `$s0` (con un desplazamiento de 0 bytes), se extrae el número de 32 bits allí guardado y se coloca en el registro `$t1`.

9. SW (Store Word)

Instrucción: Copia un dato de 32 bits desde un registro hacia una dirección específica de la memoria RAM.

```
1 sw $t1, 4($s0)     # Guardar en memoria (Desplazamiento 4)
```

Explicación: Se toma el valor numérico de 32 bits que está dentro del registro `$t1` y se guarda en la memoria RAM, específicamente en la dirección apuntada por `$s0` más un desplazamiento físico de 4 bytes.

10. LH (Load Halfword)

Instrucción: Lee únicamente 16 bits (media palabra) desde la memoria RAM y los transfiere a un registro, extendiendo automáticamente el signo del número para rellenar los 32 bits del registro.

```
1 lh $t2, 0($s0)     # Cargar fraccion de 16 bits
```

Explicación: Se extrae un fragmento de 16 bits desde la dirección de memoria indicada por `$s0` y se almacena en el registro `$t2`, respetando si el número es positivo o negativo.

11. SH (Store Halfword)

Instrucción: Guarda los 16 bits inferiores (la mitad menos significativa) de un registro dentro de una posición en la memoria RAM.

```
1 sh $t2, 0($s1)     # Guardar solamente 16 bits
```

Explicación: Se extraen solo los 16 bits más bajos del registro `$t2` y se almacenan en la ubicación de memoria RAM que determina el registro `$s1`.

12. LB (Load Byte)

Instrucción: Extrae un único byte (8 bits) desde la memoria RAM y lo coloca en un registro, extendiendo el signo para ocupar los 32 bits.

```
1 lb $t3, 0($s0) # Cargar un caracter individual
```

Explicación: Se lee un byte desde la memoria RAM usando la dirección guardada en \$s0 y se transfiere al registro \$t3. Esta operación es común al procesar caracteres individuales en formato ASCII.

13. SB (Store Byte)

Instrucción: Almacena únicamente el byte inferior (los 8 bits de menor peso) de un registro en una dirección de la memoria RAM.

```
1 sb $t3, 1($s1) # Escribir 1 simple byte
```

Explicación: Se toman los 8 bits finales del registro \$t3 y se graban en la memoria RAM, en la dirección de \$s1 más un desplazamiento de 1 byte. Es de utilidad para modificar caracteres específicos o variables booleanas.

14. LUI (Load Upper Immediate)

Instrucción: Carga una constante de 16 bits en la mitad superior de un registro, mientras que la mitad inferior se llena automáticamente con ceros.

```
1 lui $t0, 0xABCD # Cargar en la parte alta (Upper)
```

Explicación: El valor hexadecimal 0xABCD se posiciona en los 16 bits más altos del registro \$t0. El registro queda con la estructura interna 0xABCD0000.

C. Operaciones Lógicas

15. AND (Y Lógico)

Instrucción: Realiza una comparación lógica "Y" bit a bit entre dos registros. El bit resultante será 1 si, y solo si, los dos bits evaluados son 1.

```
1 and $t0, $t1, $t2 # Operacion AND entre registros
```

Explicación: Se comparan uno a uno los bits de \$t1 con los de \$t2. Si ambos bits en la misma posición son 1, el bit correspondiente en \$t0 se establece en 1; de lo contrario, se establece en 0.

16. ANDI (Y Lógico Inmediato)

Instrucción: Realiza la operación lógica "Y" bit a bit entre el contenido de un registro y un valor constante directo.

```
1 andi $t0, $t1, 0xFF # Operacion AND con un numero directo
```

Explicación: Se aplica la operación AND entre el registro \$t1 y la constante hexadecimal 0xFF. El resultado se guarda en \$t0, lo cual permite aislar u ocultar bits específicos mediante un valor conocido.

17. OR (O Lógico)

Instrucción: Realiza una comparación lógica 'O' bit a bit entre dos registros. El bit resultante será 1 si al menos uno de los dos bits evaluados es 1.

```
or $t0, $t1, $t2    # Operacion OR entre registros
```

Explicación: Se evalúan los bits de \$t1 y \$t2. Si en una posición cualquiera alguno de los dos bits es 1 (o ambos), el bit en esa posición del registro \$t0 se enciende en 1.

18. ORI (O Lógico Inmediato)

Instrucción: Realiza una operación lógica 'O' bit a bit entre un registro y un valor constante directo.

```
ori $t0, $t1, 0x0F  # Operacion OR con una base constante
```

Explicación: Se aplica la operación OR entre el contenido del registro \$t1 y la constante 0x0F. Esto obliga a que los últimos 4 bits del registro destino \$t0 cambien a 1, manteniendo el resto de los bits igual a como estaban en \$t1.

19. NOR (O Negado)

Instrucción: Realiza la operación lógica combinada 'O' bit a bit entre dos registros y luego invierte el resultado (operación NOT al resultado).

```
nor $t0, $t1, $zero # Invierte todos los bits de $t1
```

Explicación: Al operar el registro \$t1 con el registro constante \$zero (que siempre vale 0) mediante un NOR, se logra invertir por completo cada bit de \$t1. El resultado guardado en \$t0 es la negación exacta (NOT) de \$t1.

20. SRL (Shift Right Logical)

Instrucción: Mueve todos los bits de un registro hacia la derecha la cantidad de posiciones indicada por una constante, rellenando los espacios vacíos del lado izquierdo con ceros.

```
srl $t0, $t1, 2      # Desplazar 2 posiciones a la derecha
```

Explicación: Todos los bits contenidos en el registro \$t1 se desplazan dos posiciones hacia la derecha y el resultado se guarda en \$t0. Matemáticamente, esta acción equivale a dividir el valor entero de \$t1 entre 4 (2^2).

D. Saltos y Condicionales (Comparadores)

21. BEQ (Igual) / BNE (Diferente)

Instrucción: Comparan el contenido de dos registros. Si los registros son iguales (**beq**) o si son diferentes (**bne**), el programa interrumpe su secuencia normal y salta a una etiqueta especificada.

```
1 beq $t0, $t1, Etiqueta # Si (t0 == t1) saltar a Etiqueta
2 bne $t0, $t1, Etiqueta # Si (t0 != t1) saltar a Etiqueta
```

Explicación: En la primera línea, si el valor de **\$t0** es idéntico al de **\$t1**, la ejecución continúa en **Etiqueta**. En la segunda línea, el salto a **Etiqueta** ocurre únicamente si los valores de ambos registros son distintos.

22. BLT y BLE (Menor / Menor o igual)

Instrucción: Evalúan si un registro es menor (**blt**) o si es menor o igual (**ble**) en comparación con un segundo registro. De cumplirse la condición, el flujo salta a la etiqueta indicada.

```
1 blt $t0, $t1, Etiqu # Si (t0 < t1) saltar a Etiqu
2 ble $t0, $t1, Etiqu # Si (t0 <= t1) saltar a Etiqu
```

Explicación: El programa redirige su flujo hacia **Etiqueta** si el valor numérico almacenado en **\$t0** resulta ser menor estricto o menor/igual que el valor almacenado en **\$t1**.

23. BGT y BGE (Mayor / Mayor o igual)

Instrucción: Evalúan si un registro es mayor (**bgt**) o si es mayor o igual (**bge**) en comparación con un segundo registro. De cumplirse la condición, el flujo salta a la etiqueta indicada.

```
1 bgt $t0, $t1, Etiqu # Si (t0 > t1) saltar a Etiqu
2 bge $t0, $t1, Etiqu # Si (t0 >= t1) saltar a Etiqu
```

Explicación: El programa altera su orden de ejecución saltando a **Etiqueta** si el valor numérico del registro **\$t0** es mayor estricto o mayor/igual que el contenido del registro **\$t1**.

24. SLT / SLTI (Set Less Than)

Instrucción: Realiza una comparación entre dos valores. Si el primer registro es menor que el segundo registro (**slt**) o menor que una constante (**slti**), asigna un valor de 1 (verdadero) en el registro destino. En caso contrario, asigna un 0 (falso).

```
1 slt $t0, $s1, $s2 # Guarda un 1 si $s1 < $s2
2 slti $t0, $s1, 10 # Guarda un 1 si $s1 < 10 (con constante)
```

Explicación: En la primera línea, el registro **\$t0** recibirá el valor de 1 si el contenido de **\$s1** es menor que el de **\$s2**; de lo contrario recibirá un 0. En la segunda línea ocurre lo mismo, pero comparando **\$s1** contra la constante numérica 10.

E. Saltos Incondicionales (Directos)

25. J (Jump)

Instrucción: Desvía el flujo del programa de forma directa y obligatoria hacia una etiqueta específica, sin realizar ninguna comparación previa.

```
1 j Fin_bucle           # Salta directamente a la etiqueta Fin_bucle
```

Explicación: La ejecución del código interrumpe inmediatamente su orden lineal y se transfiere de forma directa a la línea donde se encuentra la etiqueta `Fin_bucle`.

26. JAL (Jump And Link)

Instrucción: Realiza un salto incondicional hacia una etiqueta (normalmente una subrutina o función) y, simultáneamente, guarda la dirección de la siguiente instrucción en el registro de retorno (`$ra`).

```
1 jal FuncionA          # Llama a la funcion y guarda la direccion de  
    retorno
```

Explicación: El programa salta a ejecutar el bloque de código denominado `FuncionA`. Antes de efectuar el salto, el procesador almacena de forma automática la ubicación del punto de origen en el registro `$ra`, permitiendo que el programa sepa a dónde regresar después.

27. JR (Jump Register)

Instrucción: Efectúa un salto incondicional redirigiendo el flujo del programa a la dirección de memoria exacta que se encuentra almacenada dentro de un registro específico.

```
1 jr $ra                # Regresar a quien llamo a esta funcion
```

Explicación: El procesador lee la dirección de memoria guardada en el registro `$ra` (colocada previamente por una instrucción `jal`) y regresa el flujo del programa exactamente a la línea posterior desde donde se invocó la subrutina.

F. Llamadas al Sistema (SYSCALL)

Funciones del Sistema (Input y Consola)

La instrucción `syscall` detiene momentáneamente el programa para solicitarle al simulador o al sistema operativo que realice una tarea de entrada o salida. El procesador sabrá exactamente qué acción ejecutar dependiendo del número entero que se haya cargado previamente en el registro `$v0`.

La lista principal de acciones que el sistema puede ejecutar es la siguiente:

- `li $v0, 1: Imprimir un Entero.` Muestra en la pantalla de la consola el número entero almacenado en el registro de argumento `$a0`.
- `li $v0, 4: Imprimir un Texto / String.` Muestra en pantalla la cadena de caracteres cuya dirección de memoria esté almacenada en el registro `$a0`.
- `li $v0, 5: Leer un Entero.` Detiene el programa y espera a que el usuario ingrese un número por teclado. El valor introducido se almacena automáticamente en el registro `$v0`.
- `li $v0, 8: Leer un String.` Espera a que el usuario introduzca texto por teclado y lo almacena en un espacio reservado de la memoria RAM.
- `li $v0, 10: Finalizar el Programa.` Termina la ejecución del código de forma correcta y segura.

Ejemplo General en Programa: Imprimir un Texto

La metodología en el código sigue siempre el mismo orden: primero se le indica al registro `$v0` el código de la acción, luego se preparan los argumentos necesarios en `$a0`, y finalmente se llama a `syscall` para ejecutar:

```
.data
    mensaje1: .asciiz "Demostracion Hola Mundo\n"

.text
main:
    # ---- PASO 1: IMPRESION DE TEXTO ----
    li $v0, 4           # Se carga el codigo 4 (Imprimir String)
                        en $v0
    la $a0, mensaje1    # Se carga la direccion del texto en $a0
    syscall             # Se ejecuta la llamada para mostrar el
                        texto en pantalla

    # ---- PASO 2: FINALIZAR PROGRAMA ----
    li $v0, 10          # Se carga el codigo 10 (Salir del
                        programa) en $v0
    syscall             # Se ejecuta la llamada para cerrar el
                        simulador MIPS
```

G. Lógica Básica de Ciclos (FOR y WHILE)

Para que sea más fácil de entender, la forma en que MIPS construye sus ciclos iterativos se puede comparar directamente con la sintaxis clásica de lenguajes como C/C++.

Ciclo Básico WHILE

En MIPS32, la estructura de un bucle se construye utilizando una **condición de salida inversa** a la planteada en lenguajes de alto nivel. Si la condición inversa se cumple, el programa realiza un salto condicional para escapar de las iteraciones. Al final del bloque interno del ciclo, se añade un salto incondicional (j) para regresar al inicio y volver a evaluar.

Interpretación equivalente (Sintaxis C):

```
int i = 0;
while (i < 10) {           // La logica es repetir mientras se cumpla
    (i < 10)
    // Bloque de instrucciones internas
    i++;
}
```

Equivalencia construida sobre simulador MIPS32:

```
li $t0, 0                  # Se inicializa el contador i = 0

Ciclo_While:
    bge $t0, 10, Fin_While # Condicion inversa: si i >= 10,
                           # salta fuera del ciclo

    # ... Bloque de instrucciones MIPS a repetir ...

    addi $t0, $t0, 1        # Incremento del contador (i++)
    j Ciclo_While           # Salto incondicional para repetir la
                           # comprobacion

Fin_While:
    # Continuacion logica del programa fuera del ciclo
```

Ciclo Básico FOR

Para la construcción de un ciclo **for** se requiere definir explícitamente un registro que sirva como índice (inicializador) y otro registro que actúe como el valor límite o tope. En este formato común, se utiliza la instrucción **beq** para comprobar si el índice ha alcanzado exactamente el valor límite, provocando la salida del bucle cuando la condición se cumple.

Interpretación equivalente (Sintaxis C):

```
for (int i = 0; i != 5; i++) {  
    // Bloque de instrucciones a repetir  
}
```

Equivalencia construida sobre simulador MIPS32:

```
li $t0, 0                # Se define el registro de control i  
    = 0  
li $t1, 5                # Se define el valor limite o tope =  
    5  
  
Ciclo_For:  
    beq $t0, $t1, Fin_For # Si el indice es igual al tope (i ==  
        5), termina el ciclo  
  
    # ... Instrucciones y funciones a repetir ...  
  
    addi $t0, $t0, 1      # Incremento del paso del ciclo (i++)  
    j Ciclo_For          # Salto incondicional para volver a  
        evaluar el ciclo  
  
Fin_For:  
    # Continuacion del programa tras finalizar el bucle FOR
```